

— THE 4 LEVELS OF —

PYTHON

EVERY DATA ANALYST
SHOULD KNOW

— • —

Your roadmap from basics to advanced analysis



LEVEL 1

PYTHON FUNDAMENTALS



Learn the basics you need to work with Python



1

VARIABLES & DATA TYPES

Store and work with different types of data.

```
x = 10
name = "Data Analyst"
price = 19.99
is_active = True
type(x) # int
```



2

LISTS & DICTIONARIES

Organize data using lists and dictionaries.

```
fruits = ["apple", "banana", "orange"]
info = {
    "name": "Ali",
    "role": "Analyst",
    "exp": 2
}
```



3

IF / ELSE

Make decisions in your code.

```
age = 18
if age >= 18:
    print("Adult")
else:
    print("Minor")
```



4

FOR LOOPS

Iterate over sequences and perform actions.

```
fruits = ["apple", "banana", "orange"]
for fruit in fruits:
    print(fruit)
```



5

FUNCTIONS

Write reusable blocks of code.

```
def greet(name):
    return f"Hello, {name}!"

print(greet("Ali"))
```



6

INPUT / OUTPUT

Get input from users and display output.

```
name = input("Enter your name: ")
print("Hello, " + name)
```



7

COMMENTS

Add comments to make your code readable.

```
# This is a single line comment
x = 10

# This is
# a multi-line comment
print(x)
```



8

BASIC OPERATORS

Use operators for calculations and logic.

```
a = 10
b = 3
a + b # 13
a - b # 7
a * b # 30
a / b # 3.33
a % b # 1
```



Master these basics to build a **strong foundation** for data analysis with Python.



LEVEL 2

DATA HANDLING



Learn to work with and manipulate data



1

NUMPY

Work with arrays and perform mathematical operations.

```
import numpy as np

arr = np.array([1, 2, 3, 4])

arr_mean = np.mean(arr)
arr_mean # 2.5
```



2

PANDAS

Powerful library for data manipulation and analysis.

```
import pandas as pd

df = pd.read_csv('data.csv')

df.head()
```



3

DATAFRAMES

Work with structured data using rows and columns.

```
data = {
    'Name': ['Ali', 'Sara'],
    'Age': [25, 30]
}

df = pd.DataFrame(data)

df.shape # (2, 2)
```



4

FILTERING & SORTING

Filter rows and sort data easily.

```
df_filtered = df[df['Age'] > 25]

df_sorted = df.sort_values(
    by='Age', ascending=False)

df_filtered
```



5

MISSING VALUES

Detect and handle missing data.

```
df.isnull().sum()

df_clean = df.dropna()

df_filled = df.fillna(0)
```



6

GROUPBY & AGGREGATIONS

Group data and calculate summary statistics.

```
df.groupby('Category')['Sales']
    .agg(['sum', 'mean', 'count'])
    .reset_index()
```



7

MERGING DATA

Combine DataFrames using keys.

```
df_merged = pd.merge(
    df1, df2, on='ID', how='inner')

df_merged
```



8

JOINING DATA

Join tables using different join types.

```
df_joined = df1.join(
    df2.set_index('ID'),
    on='ID', how='left')

df_joined
```



Clean, organize, and transform data — the **key skills** of a data analyst.



LEVEL 3

DATA ANALYSIS & VISUALIZATION



Turn data into insights and visuals



1

EXPLORATORY DATA ANALYSIS (EDA)

Explore and understand your data.

```
df.head()
df.info()
df.describe(include='all')
df.shape # (rows, columns)
```



2

DESCRIPTIVE STATISTICS

Summarize and describe data numerically.

```
df['Age'].mean()
df['Age'].median()
df['Age'].std()
df['Age'].quantile([0.25,
                    0.5, 0.75])
```



3

MATPLOTLIB

Create static, interactive, and custom plots.

```
import matplotlib.pyplot as plt

plt.figure(figsize=(8, 4))
plt.plot(df['Date'], df['Sales'])
plt.title('Sales Over Time')
plt.xlabel('Date')
plt.ylabel('Sales')
plt.show()
```



4

SEABORN

Make statistical graphics with less code.

```
import seaborn as sns

sns.histplot(df['Age'],
             bins=20, kde=True)
plt.title('Age Distribution')
plt.show()
```



5

DISTRIBUTIONS

Understand the distribution of your data.

```
sns.histplot(df['Income'],
             bins=30, kde=True)
plt.axvline(df['Income'].mean(),
            color='red', linestyle='--')
plt.show()
```



6

CORRELATION

Measure relationships between variables.

```
corr = df.corr(numeric_only=True)
sns.heatmap(corr, annot=True,
            cmap='coolwarm',
            fmt='.2f')
plt.title('Correlation Matrix')
plt.show()
```



7

OUTLIER ANALYSIS

Detect and understand outliers in data.

```
sns.boxplot(x='Category',
            y='Amount', data=df)
plt.title('Outliers by Category')
plt.show()
```



8

DATA VISUALIZATION BEST PRACTICES

Build clear, meaningful, and effective visuals.

```
# Good visuals are:
# 1. Simple
# 3. Clear
# 3. Accurate
# 4. Focused on the insight
```



Explore your data, find patterns, and **communicate insights** with powerful visuals.



LEVEL 4

REAL-WORLD DATA ANALYSIS



Apply Python to real analytical problems



1

DATA CLEANING WORKFLOWS

Build clean, reliable datasets ready for analysis.

```
df = df.drop_duplicates()

df['Date'] = pd.to_datetime(df['Date'])

df = df.fillna(df.median(numeric_only=True))

df = df.dropna(subset=['Sales'])
```



2

ADVANCED PANDAS

Use advanced techniques to manipulate data.

```
df.assign(Tax=lambda x: x['Sales']
          * 0.1)

df.pivot_table(index='Region',
               columns='Product',
               values='Sales', aggfunc='sum')

df.sort_values(['Region', 'Sales'],
               ascending=[True, False])
```



3

TIME SERIES ANALYSIS

Analyze data over time and forecast future trends.

```
df = df.set_index('Date').sort_index()
df_resampled = df['Sales'].resample('M')
    .sum()

from statsmodels.tsa.arima.model
import ARIMA

model = ARIMA(df_resampled, order=(1,1,1))
forecast = model.fit().forecast(12)
```



4

STATISTICAL ANALYSIS

Apply statistical tests to extract meaningful insights.

```
from scipy import stats

# Correlation
corr, p = stats.pearsonr(df['X'],
                        df['Y'])

# T-test
t_stat, p_value = stats.ttest_ind(
    group1, group2, equal_var=False)
```



5

AUTOMATED REPORTS

Generate automated reports and summaries.

```
import pandas as pd
from jinja2 import Template

report = Template(open('report.html').read())
html_out = report.render(data=df.to_dict())
open('report.html', 'w').write(html_out)
```



6

WORKING WITH APIS

Extract data from APIs and use it in your analysis.

```
import requests

url = 'https://api.example.com/data'
response = requests.get(url)
data = response.json()
df = pd.DataFrame(data['items'])
```



7

SQL + PYTHON INTEGRATION

Use Python with SQL databases to analyze data at scale.

```
import pandas as pd
from sqlalchemy import create_engine

engine = create_engine('sqlite:///data.db')
query = """
SELECT * FROM sales
WHERE year = 2024
"""
df = pd.read_sql(query, engine)
```



8

SOLVE BUSINESS PROBLEMS

Combine everything to answer real business questions.

```
# Example Workflow
df = load_data()
df = clean_data(df)
df = analyze(df)
insights = generate_insights(df)
create_report(insights)
```



Combine your skills, automate your workflow, and turn data into **real business impact**.

